

Математическое моделирование

Лабораторная работа № 4

Гафоров Нурмухаммад

Содержание

1	Цель работы	6
2	Задание	7
3	Выполнение лабораторной работы	8
3.1	Теоретические сведения	8
3.2	Задача	9
3.3	Начальные условия и временной интервал	11
3.4	Первая модель: незатухающие колебания	11
3.5	Вторая модель: затухающие колебания	12
3.6	Третья модель: затухающие колебания с внешним воздействием	12
3.7	Утилита для решения, построения графиков и сохранения результатов	13
3.8	Запуск первой модели	14
3.9	Запуск второй модели	15
3.10	Запуск третьей модели	16
4	Параметрическое исследование трех моделей ОДУ	17
4.1	Активация проекта и загрузка пакетов	17
4.2	Установка каталогов	17
4.3	Внешнее воздействие для третьей модели	18
4.4	Определение моделей	18
4.5	Определение базовых параметров	19
4.6	Функция-обертка для запуска одного эксперимента	20
4.7	Запуск базового эксперимента для первой модели	22
4.8	Визуализация базового эксперимента для первой модели	22
4.9	Базовый эксперимент для второй модели	23
4.10	Базовый эксперимент для третьей модели	25
4.11	Параметрическое сканирование	27
4.12	Запуск всех экспериментов и сбор результатов	28
4.13	Анализ и визуализация результатов сканирования	30
4.14	Бенчмаркинг	34
4.15	Сохранение таблиц результатов	37
4.16	Сохранение всех результатов	38
4.17	Базовые эксперименты	39
4.18	Параметрическое сканирование	45

4.19 Время вычислений	47
4.20 Анализ метрики	47
5 Выводы	49
Список литературы	50

Список иллюстраций

4.1	Первая модель: зависимость $y(t)$	39
4.2	Первая модель: фазовый портрет	40
4.3	Вторая модель: зависимость $y(t)$	41
4.4	Вторая модель: фазовый портрет	42
4.5	Третья модель: зависимость $y(t)$	43
4.6	Третья модель: фазовый портрет	44
4.7	Сканирование траекторий $x(t)$	45
4.8	Сканирование траекторий $y(t)$	46
4.9	Зависимость времени вычисления	47

Список таблиц

1 Цель работы

Исследовать математическую модель гармонического осциллятора и проанализировать её поведение в различных условиях.

2 Задание

1. Получить и визуализировать решение уравнения гармонического осциллятора без затухания
2. Записать и исследовать уравнение затухающего осциллятора, включая фазовый портрет
3. Рассмотреть случай воздействия внешней силы, построить решение и фазовый портрет

3 Выполнение лабораторной работы

3.1 Теоретические сведения

Ряд физических процессов — колебания груза на пружине, движение маятника, процессы в электрических цепях — можно описать единым дифференциальным уравнением. Это уравнение лежит в основе модели линейного гармонического осциллятора.

Общее уравнение движения имеет вид:

$$\ddot{x} + 2\gamma\dot{x} + \omega_0^2 x = 0$$

где x — координата системы, γ — коэффициент затухания, ω_0 — собственная частота.

Данное уравнение относится к линейным дифференциальным уравнениям второго порядка и описывает динамику системы.

Если потери энергии отсутствуют ($\gamma = 0$), уравнение принимает форму:

$$\ddot{x} + \omega_0^2 x = 0$$

Для получения единственного решения необходимо задать начальные условия:

$$\begin{cases} x(t_0) = x_0 \\ \dot{x}(t_0) = y_0 \end{cases}$$

Перепишем уравнение в виде системы:

$$\begin{cases} \dot{x} = y \\ \dot{y} = -\omega_0^2 x \end{cases}$$

Начальные условия:

$$\begin{cases} x(t_0) = x_0 \\ y(t_0) = y_0 \end{cases}$$

Переменные x и y формируют фазовое пространство. Каждое решение системы соответствует траектории на фазовой плоскости. Совокупность таких траекторий образует фазовый портрет, отражающий общее поведение системы.

3.2 Задача

Необходимо построить решения и фазовые портреты для следующих моделей:

1. Без затухания и внешней силы

$$\ddot{x} + 5.2x = 0$$

2. С затуханием без внешнего воздействия

$$\ddot{x} + 14\dot{x} + 0.5x = 0$$

3. С затуханием и внешней силой

$$\ddot{x} + 13\dot{x} + 0.3x = 0.8 \sin(9t)$$

Параметры моделирования:

$t \in [0; 59]$, шаг 0.05, $x_0 = 0.5$, $y_0 = -1.5$

3.2.1 Преобразование моделей

1. Без затухания:

$$\begin{cases} \dot{x} = y \\ \dot{y} = -\omega_0^2 x \end{cases}$$

2. С затуханием:

$$\begin{cases} \dot{x} = y \\ \dot{y} = -2\gamma y - \omega_0^2 x \end{cases}$$

3. С внешней силой:

$$\begin{cases} \dot{x} = y \\ \dot{y} = F(t) - 2\gamma y - \omega_0^2 x \end{cases}$$

Для численного моделирования применялись внешние программные модули:

```
using DrWatson
@quickactivate "project"
```

```

using DifferentialEquations
using Plots
using DataFrames
using JLD2

script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE))
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

```

3.3 Начальные условия и временной интервал

```

x0 = 0.5
y0 = -1.5
u0 = [x0, y0]

t0 = 0.0
tmax = 59.0
tspan = (t0, tmax)
tgrid = collect(LinRange(t0, tmax, 1000))

```

3.4 Первая модель: незатухающие колебания

```

w1 = 5.2
p1 = (w = w1,)

function syst1!(dy, y, p, t)

```

```
dy[1] = y[2]
dy[2] = -p.w * y[1]
end
```

3.5 Вторая модель: затухающие колебания

```
w2 = 0.5
g2 = 14.0
p2 = (w = w2, g = g2)

function syst2!(dy, y, p, t)
    dy[1] = y[2]
    dy[2] = -p.g * y[2] - p.w * y[1]
end
```

3.6 Третья модель: затухающие колебания с внешним воздействием

```
w3 = 0.3
g3 = 13.0
p3 = (w = w3, g = g3)

function P(t)
    return 0.8 * sin(9 * t)
end
```

```

function syst3!(dy, y, p, t)
    dy[1] = y[2]
    dy[2] = -p.g * y[2] - p.w * y[1] + P(t)
end

```

3.7 Утилита для решения, построения графиков и сохранения результатов

```

function run_model(case_name; model!, u0, tspan, p, tgrid, vel_file, phase_file)
    prob = ODEProblem(model!, u0, tspan, p)
    sol = solve(prob, Tsit5(), saveat = tgrid)

    df = DataFrame(
        t = sol.t,
        x = getindex.(sol.u, 1),
        y = getindex.(sol.u, 2)
    )

    plt1 = plot(
        sol,
        idxs = (2),
        xlabel = "t",
        ylabel = "y(t)",
        title = "Модель $case_name: зависимость y(t)",
        label = "y(t)",
        lw = 2,
        legend = :topright
    )

```

```

)
savefig(plt1, plotsdir(script_name, vel_file))

plt2 = plot(
    sol,
    idxs = (1, 2),
    xlabel = "x",
    ylabel = "y",
    title = "Модель $case_name: фазовый портрет",
    label = "y(x)",
    lw = 2,
    legend = :topright
)
savefig(plt2, plotsdir(script_name, phase_file))

@save datadir(script_name, "data_$(case_name).jld2") df p u0 tspan tgrid

return (sol = sol, df = df, plt_time = plt1, plt_phase = plt2)
end

```

3.8 Запуск первой модели

```

res1 = run_model(
    "1";
    model! = syst1!,
    u0 = u0,
    tspan = tspan,

```

```

    p = p1,
    tgrid = tgrid,
    vel_file = "01j.png",
    phase_file = "02j.png"
)

println("Модель 1 – первые 5 строк таблицы:")
println(first(res1.df, 5))

```

3.9 Запуск второй модели

```

res2 = run_model(
    "2";
    model! = syst2!,
    u0 = u0,
    tspan = tspan,
    p = p2,
    tgrid = tgrid,
    vel_file = "03j.png",
    phase_file = "04j.png"
)

println("\nМодель 2 – первые 5 строк таблицы:")
println(first(res2.df, 5))

```

3.10 Запуск третьей модели

```
res3 = run_model(  
    "3";  
    model! = syst3!,  
    u0 = u0,  
    tspan = tspan,  
    p = p3,  
    tgrid = tgrid,  
    vel_file = "05j.png",  
    phase_file = "06j.png"  
)  
  
println("\nМодель 3 – первые 5 строк таблицы:")  
println(first(res3.df, 5))
```

(опционально) показать последний график в интерактивной среде

```
res3.plt_phase
```

4 Параметрическое исследование трех моделей ОДУ

4.1 Активация проекта и загрузка пакетов

```
using DrWatson
@quickactivate "project"

using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
```

4.2 Установка каталогов

```
script_name = isempty(PROGRAM_FILE) ? "interactive" : splitext(basename(PROGRAM_FILE))
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

4.3 Внешнее воздействие для третьей модели

```
function P(t)
    return 0.8 * sin(9 * t)
end
```

4.4 Определение моделей

Первая модель: $\dot{x} = y$ $\dot{y} = -w \cdot x$

```
function syst_model1!(dy, y, p, t)
    dy[1] = y[2]
    dy[2] = -p.w * y[1]
end
```

Вторая модель: $\dot{x} = y$ $\dot{y} = -g \cdot y - w \cdot x$

```
function syst_model2!(dy, y, p, t)
    dy[1] = y[2]
    dy[2] = -p.g * y[2] - p.w * y[1]
end
```

Третья модель: $\dot{x} = y$ $\dot{y} = -g \cdot y - w \cdot x + P(t)$

```
function syst_model3!(dy, y, p, t)
    dy[1] = y[2]
    dy[2] = -p.g * y[2] - p.w * y[1] + P(t)
end
```

4.5 Определение базовых параметров

model_type управляет выбором системы: :model1 -> первая система :model2
-> вторая система :model3 -> третья система

```
base_params = Dict(  
    :x0 => 0.5,  
    :y0 => -1.5,  
  
    :tspan => (0.0, 59.0),  
    :nt => 1000,  
  
    :model_type => :model1,
```

Параметры первой модели

```
:w1 => 5.2,
```

Параметры второй модели

```
:w2 => 0.5,  
:g2 => 14.0,
```

Параметры третьей модели

```
:w3 => 0.3,  
:g3 => 13.0,  
  
:solver => Tsit5(),  
:experiment_name => "base_experiment"  
)
```

```
println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println(" $key = $value")
end
```

4.6 Функция-обертка для запуска одного эксперимента

```
function run_single_experiment(params::Dict)
    @unpack x0, y0, tspan, nt, model_type, w1, w2, g2, w3, g3, solver = params

    u0 = [x0, y0]
    tgrid = collect(LinRange(tspan[1], tspan[2], nt))

    if model_type == :model1
        p = (w = w1,)
        prob = ODEProblem(syst_model1!, u0, tspan, p)
    elseif model_type == :model2
        p = (w = w2, g = g2)
        prob = ODEProblem(syst_model2!, u0, tspan, p)
    else
        p = (w = w3, g = g3)
        prob = ODEProblem(syst_model3!, u0, tspan, p)
    end

    sol = solve(prob, solver; saveat = tgrid)
```

```
x_vals = first.(sol.u)
y_vals = last.(sol.u)
```

— Мини-анализ —

```
x_final = x_vals[end]
y_final = y_vals[end]

x_max_abs = maximum(abs.(x_vals))
y_max_abs = maximum(abs.(y_vals))

norm_final = sqrt(x_final^2 + y_final^2)

return Dict(
    "solution" => sol,
    "t_points" => sol.t,
    "x_values" => x_vals,
    "y_values" => y_vals,

    "x_final" => x_final,
    "y_final" => y_final,
    "x_max_abs" => x_max_abs,
    "y_max_abs" => y_max_abs,
    "norm_final" => norm_final,

    "parameters" => params
)
end
```

4.7 Запуск базового эксперимента для первой модели

```
data_base1, path_base1 = produce_or_load(
    datadir(script_name, "single"),
    base_params,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты базового эксперимента для первой модели:")
println(" x_final: ", data_base1["x_final"])
println(" y_final: ", data_base1["y_final"])
println(" x_max_abs: ", data_base1["x_max_abs"])
println(" y_max_abs: ", data_base1["y_max_abs"])
println(" norm_final: ", round(data_base1["norm_final"]; digits = 4))
println(" Файл результатов: ", path_base1)
```

4.8 Визуализация базового эксперимента для первой модели

```
p1 = plot(
    data_base1["t_points"], data_base1["y_values"],
    label = "y(t)",
    xlabel = "t",
    ylabel = "y",
```

```

    title = "Первая модель: зависимость  $y(t)$ ",
    lw = 2,
    legend = :topright,
    grid = true
)
savefig(p1, plotsdir(script_name, "01j.png"))

p2 = plot(
    data_base1["x_values"], data_base1["y_values"],
    label = "Фазовая траектория",
    xlabel = "x",
    ylabel = "y",
    title = "Первая модель: фазовый портрет",
    lw = 2,
    legend = :topright,
    grid = true
)
savefig(p2, plotsdir(script_name, "02j.png"))

```

4.9 Базовый эксперимент для второй модели

```

base_params2 = copy(base_params)
base_params2[:model_type] = :model2
base_params2[:experiment_name] = "base_experiment_model2"

data_base2, path_base2 = produce_or_load(
    datadir(script_name, "single"),

```

```

    base_params2,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты базового эксперимента для второй модели:")
println(" x_final: ", data_base2["x_final"])
println(" y_final: ", data_base2["y_final"])
println(" x_max_abs: ", data_base2["x_max_abs"])
println(" y_max_abs: ", data_base2["y_max_abs"])
println(" norm_final: ", round(data_base2["norm_final"]; digits = 4))
println(" Файл результатов: ", path_base2)

p3 = plot(
    data_base2["t_points"], data_base2["y_values"],
    label = "y(t)",
    xlabel = "t",
    ylabel = "y",
    title = "Вторая модель: зависимость y(t)",
    lw = 2,
    legend = :topright,
    grid = true
)

savefig(p3, plotsdir(script_name, "03j.png"))

p4 = plot(

```

```

    data_base2["x_values"], data_base2["y_values"],
    label = "Фазовая траектория",
    xlabel = "x",
    ylabel = "y",
    title = "Вторая модель: фазовый портрет",
    lw = 2,
    legend = :topright,
    grid = true
)
savefig(p4, plotsdir(script_name, "04j.png"))

```

4.10 Базовый эксперимент для третьей модели

```

base_params3 = copy(base_params)
base_params3[:model_type] = :model3
base_params3[:experiment_name] = "base_experiment_model3"

data_base3, path_base3 = produce_or_load(
    datadir(script_name, "single"),
    base_params3,
    run_single_experiment;
    prefix = "ode",
    tag = false,
    verbose = true
)

println("\nРезультаты базового эксперимента для третьей модели:")

```

```

println(" x_final: ", data_base3["x_final"])
println(" y_final: ", data_base3["y_final"])
println(" x_max_abs: ", data_base3["x_max_abs"])
println(" y_max_abs: ", data_base3["y_max_abs"])
println(" norm_final: ", round(data_base3["norm_final"]; digits = 4))
println(" Файл результатов: ", path_base3)

p5 = plot(
  data_base3["t_points"], data_base3["y_values"],
  label = "y(t)",
  xlabel = "t",
  ylabel = "y",
  title = "Третья модель: зависимость y(t)",
  lw = 2,
  legend = :topright,
  grid = true
)
savefig(p5, plotsdir(script_name, "05j.png"))

p6 = plot(
  data_base3["x_values"], data_base3["y_values"],
  label = "Фазовая траектория",
  xlabel = "x",
  ylabel = "y",
  title = "Третья модель: фазовый портрет",
  lw = 2,
  legend = :topright,
  grid = true
)

```

```
)  
savefig(p6, plotsdir(script_name, "06j.png"))
```

4.11 Параметрическое сканирование

Сканируем основной параметр: для model1 -> w1 для model2 -> w2 для model3
-> w3

```
param_grid = Dict(  
    :x0 => [0.5],  
    :y0 => [-1.5],  
  
    :tspan => [(0.0, 59.0)],  
    :nt => [1000],  
  
    :model_type => [:model1, :model2, :model3],  
  
    :w1 => [3.0, 5.2, 7.0],  
    :w2 => [0.2, 0.5, 0.8],  
    :g2 => [14.0],  
    :w3 => [0.1, 0.3, 0.6],  
    :g3 => [13.0],  
  
    :solver => [Tsit5()],  
    :experiment_name => ["parametric_scan"]  
)  
  
all_params = dict_list(param_grid)
```

```

println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые model_type: ", param_grid[:model_type])
println("Исследуемые w1: ", param_grid[:w1])
println("Исследуемые w2: ", param_grid[:w2])
println("Исследуемые w3: ", param_grid[:w3])
println("="^60)

```

4.12 Запуск всех экспериментов и сбор результатов

```

all_results = []
all_dfs = []

for (i, params) in enumerate(all_params)
    println(
        "Прогресс: $i/$(length(all_params)) | " *
        "model_type=$(params[:model_type]) | " *
        "w1=$(params[:w1]) | w2=$(params[:w2]) | w3=$(params[:w3])"
    )

    data, path = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
    )
end

```

```

        verbose = false
    )

    result_summary = merge(
        params,
        Dict(
            :x_final => data["x_final"],
            :y_final => data["y_final"],
            :x_max_abs => data["x_max_abs"],
            :y_max_abs => data["y_max_abs"],
            :norm_final => data["norm_final"],
            :filepath => path
        )
    )
    push!(all_results, result_summary)

    df = DataFrame(
        t = data["t_points"],
        x = data["x_values"],
        y = data["y_values"],
        model_type = fill(string(params[:model_type]), length(data["t_points"])),
        w1 = fill(params[:w1], length(data["t_points"])),
        w2 = fill(params[:w2], length(data["t_points"])),
        w3 = fill(params[:w3], length(data["t_points"]))
    )
    push!(all_dfs, df)
end

```

4.13 Анализ и визуализация результатов сканирования

```
results_df = DataFrame(all_results)
println("\nСводная таблица результатов (первые строки):")
println(first(results_df, 10))
```

Сравнительный график $x(t)$ для всех комбинаций

```
p7 = plot(size = (900, 520), dpi = 150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = params[:model_type] == :model1 ? "model1, w1=$(params[:w1])" :
                params[:model_type] == :model2 ? "model2, w2=$(params[:w2])" :
                "model3, w3=$(params[:w3])"

    plot!(
        p7,
        data["t_points"], data["x_values"],
        label = label_text,
        lw = 2,
```

```

        alpha = 0.8
    )
end
plot!(
    p7,
    xlabel = "t",
    ylabel = "x(t)",
    title = "Сканирование: траектории x(t) для разных параметров",
    legend = :outright,
    grid = true
)
savefig(p7, plotsdir(script_name, "parametric_scan_x_comparison.png"))

```

Сравнительный график $y(t)$ для всех комбинаций

```

p8 = plot(size = (900, 520), dpi = 150)
for params in all_params
    data, _ = produce_or_load(
        datadir(script_name, "parametric_scan"),
        params,
        run_single_experiment;
        prefix = "scan",
        tag = false,
        verbose = false
    )

    label_text = params[:model_type] == :model1 ? "model1, w1=$(params[:w1])" :
        params[:model_type] == :model2 ? "model2, w2=$(params[:w2])" :
        "model3, w3=$(params[:w3])"

```

```

plot!(
    p8,
    data["t_points"], data["y_values"],
    label = label_text,
    lw = 2,
    alpha = 0.8
)
end
plot!(
    p8,
    xlabel = "t",
    ylabel = "y(t)",
    title = "Сканирование: траектории y(t) для разных параметров",
    legend = :outright,
    grid = true
)
savefig(p8, plotsdir(script_name, "parametric_scan_y_comparison.png"))

```

График нормы финального состояния

```

p9 = plot(size = (900, 520), dpi = 150)

sub1 = results_df[results_df.model_type .== :model1, :]
sub2 = results_df[results_df.model_type .== :model2, :]
sub3 = results_df[results_df.model_type .== :model3, :]

if nrow(sub1) > 0
    plot!(
        p9,

```

```

        sub1.w1, sub1.norm_final,
        seriestype = :scatter,
        label = "model1"
    )
end

if nrow(sub2) > 0
    plot!(
        p9,
        sub2.w2, sub2.norm_final,
        seriestype = :scatter,
        label = "model2"
    )
end

if nrow(sub3) > 0
    plot!(
        p9,
        sub3.w3, sub3.norm_final,
        seriestype = :scatter,
        label = "model3"
    )
end

plot!(
    p9,
    xlabel = "Параметр модели",
    ylabel = "norm_final",

```

```

    title = "Зависимость norm_final от параметра модели",
    legend = :topleft,
    grid = true
)
savefig(p9, plotsdir(script_name, "norm_final_vs_parameter.png"))

```

4.14 Бенчмаркинг

```

println("\n" * "="^60)
println("БЕНЧМАРКИНГ ДЛЯ РАЗНЫХ ПАРАМЕТРОВ")
println("="^60)

benchmark_results = []

for params in all_params
    function benchmark_run()
        u0 = [params[:x0], params[:y0]]

        if params[:model_type] == :model1
            p = (w = params[:w1],)
            prob = ODEProblem(syst_model1!, u0, params[:tspan], p)
        elseif params[:model_type] == :model2
            p = (w = params[:w2], g = params[:g2])
            prob = ODEProblem(syst_model2!, u0, params[:tspan], p)
        else
            p = (w = params[:w3], g = params[:g3])
            prob = ODEProblem(syst_model3!, u0, params[:tspan], p)
        end
    end
end

```

```

end

return solve(
    prob,
    params[:solver];
    saveat = LinRange(params[:tspan][1], params[:tspan][2], params[:nt])
)

end

println(
    "\nБенчмарк для model_type=$(params[:model_type]), " *
    "w1=$(params[:w1]), w2=$(params[:w2]), w3=$(params[:w3]):"
)
b = @benchmark $benchmark_run() samples = 80 evals = 1
tsec = median(b).time / 1e9
println(" Медианное время: ", round(tsec; digits = 6), " сек")

push!(benchmark_results, (
    model_type = string(params[:model_type]),
    w1 = params[:w1],
    w2 = params[:w2],
    w3 = params[:w3],
    time = tsec
))

end

bench_df = DataFrame(benchmark_results)

```

График времени вычисления

```

p10 = plot(size = (900, 520), dpi = 150)

bench1 = bench_df[bench_df.model_type == "model1", :]
bench2 = bench_df[bench_df.model_type == "model2", :]
bench3 = bench_df[bench_df.model_type == "model3", :]

if nrow(bench1) > 0
  plot!(
    p10,
    bench1.w1, bench1.time,
    seriestype = :scatter,
    label = "model1"
  )
end

if nrow(bench2) > 0
  plot!(
    p10,
    bench2.w2, bench2.time,
    seriestype = :scatter,
    label = "model2"
  )
end

if nrow(bench3) > 0
  plot!(
    p10,
    bench3.w3, bench3.time,

```

```

        seriestype = :scatter,
        label = "model3"
    )
end

plot!(
    p10,
    xlabel = "Параметр модели",
    ylabel = "Время вычисления, сек",
    title = "Зависимость времени решения ODE от параметров модели",
    legend = :topleft,
    grid = true
)
savefig(p10, plotsdir(script_name, "computation_time_vs_parameter.png"))

```

4.15 Сохранение таблиц результатов

```

single_df1 = DataFrame(
    t = data_base1["t_points"],
    x = data_base1["x_values"],
    y = data_base1["y_values"]
)

single_df2 = DataFrame(
    t = data_base2["t_points"],
    x = data_base2["x_values"],
    y = data_base2["y_values"]
)

```

```
)

single_df3 = DataFrame(
    t = data_base3["t_points"],
    x = data_base3["x_values"],
    y = data_base3["y_values"]
)

all_scan_df = vcat(all_dfs...)
```

4.16 Сохранение всех результатов

```
@save datadir(script_name, "all_results.jld2") base_params base_params2 base_params3
@save datadir(script_name, "all_plots.jld2") p1 p2 p3 p4 p5 p6 p7 p8 p9 p10

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)
println("\nРезультаты сохранены в:")
println(" • data/${script_name}/single/ - базовые эксперименты")
println(" • data/${script_name}/parametric_scan/ - параметрическое сканирование")
println(" • data/${script_name}/all_results.jld2 - сводные данные")
println(" • plots/${script_name}/ - все графики")
println(" • data/${script_name}/all_plots.jld2 - объекты графиков")
println("\nДля анализа результатов используйте:")
println(" using JLD2, DataFrames")
println(" @load \"data/${script_name}/all_results.jld2\"")
```

```
println(" println(results_df)")
```

4.17 Базовые эксперименты

4.17.1 Первая модель (`model_type = model1`)

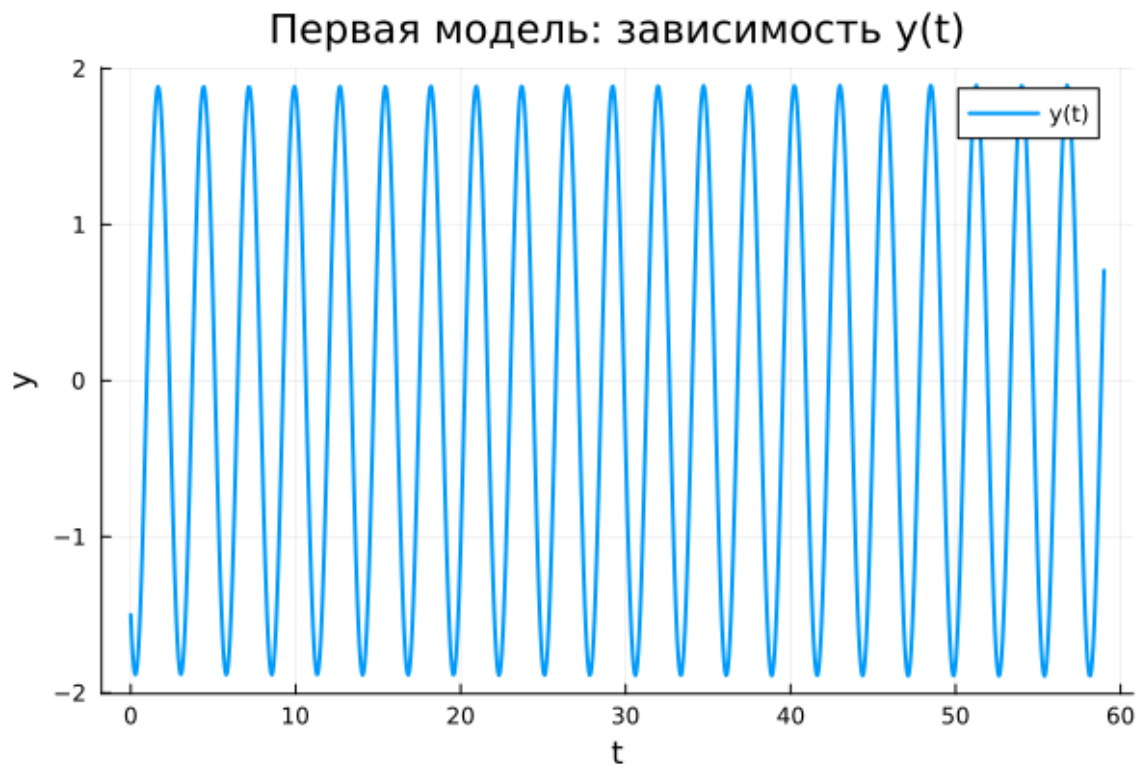


Рисунок 4.1: Первая модель: зависимость $y(t)$

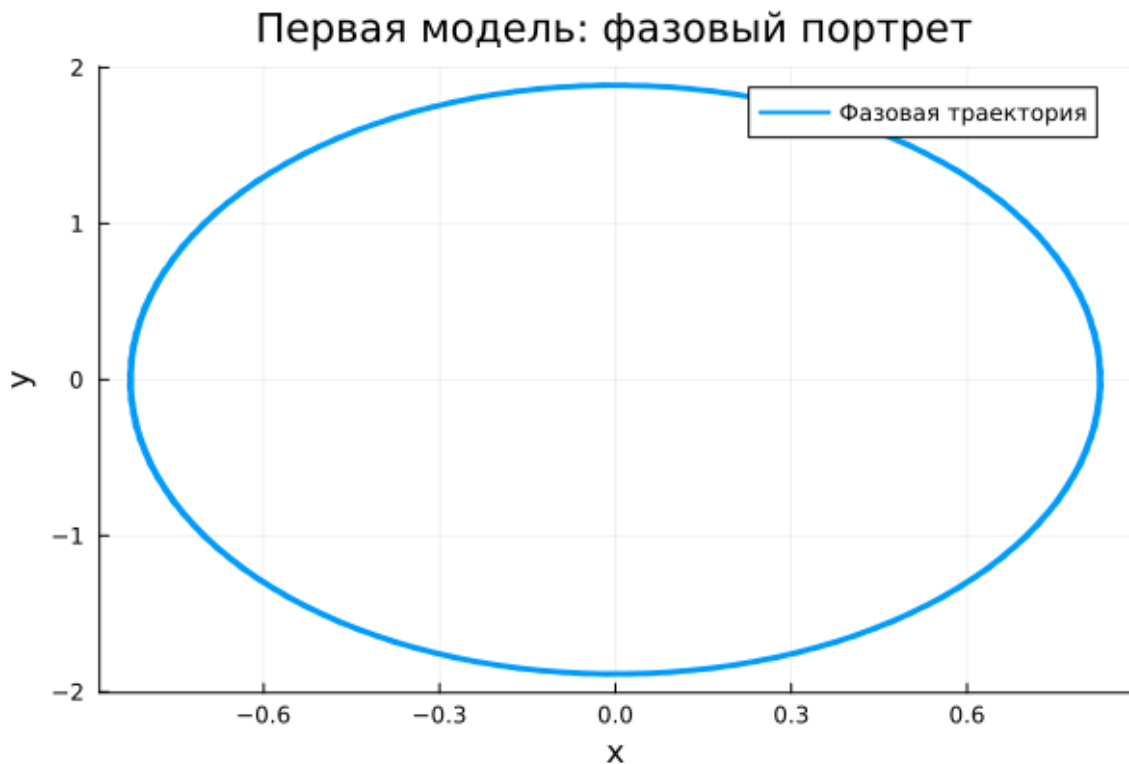


Рисунок 4.2: Первая модель: фазовый портрет

График $y(t)$ демонстрирует устойчивые периодические колебания с неизменной амплитудой. Форма сигнала остаётся регулярной на всём интервале времени.

Такое поведение характерно для консервативной системы, в которой отсутствуют потери энергии. Колебания не затухают и сохраняют свою структуру.

Фазовый портрет представляет собой замкнутую кривую, что указывает на повторяемость движения и устойчивость динамики.

Следовательно, система реализует идеальные гармонические колебания без диссипации.

4.17.2 Вторая модель (model_type = model2)

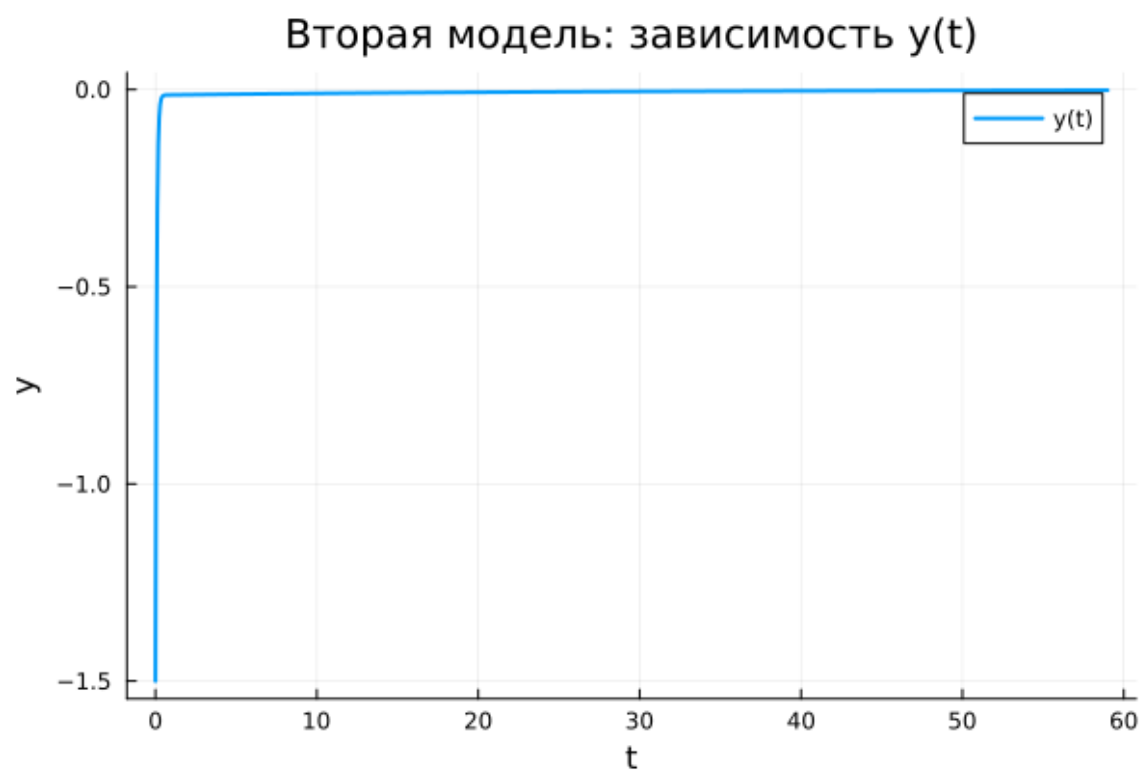


Рисунок 4.3: Вторая модель: зависимость $y(t)$

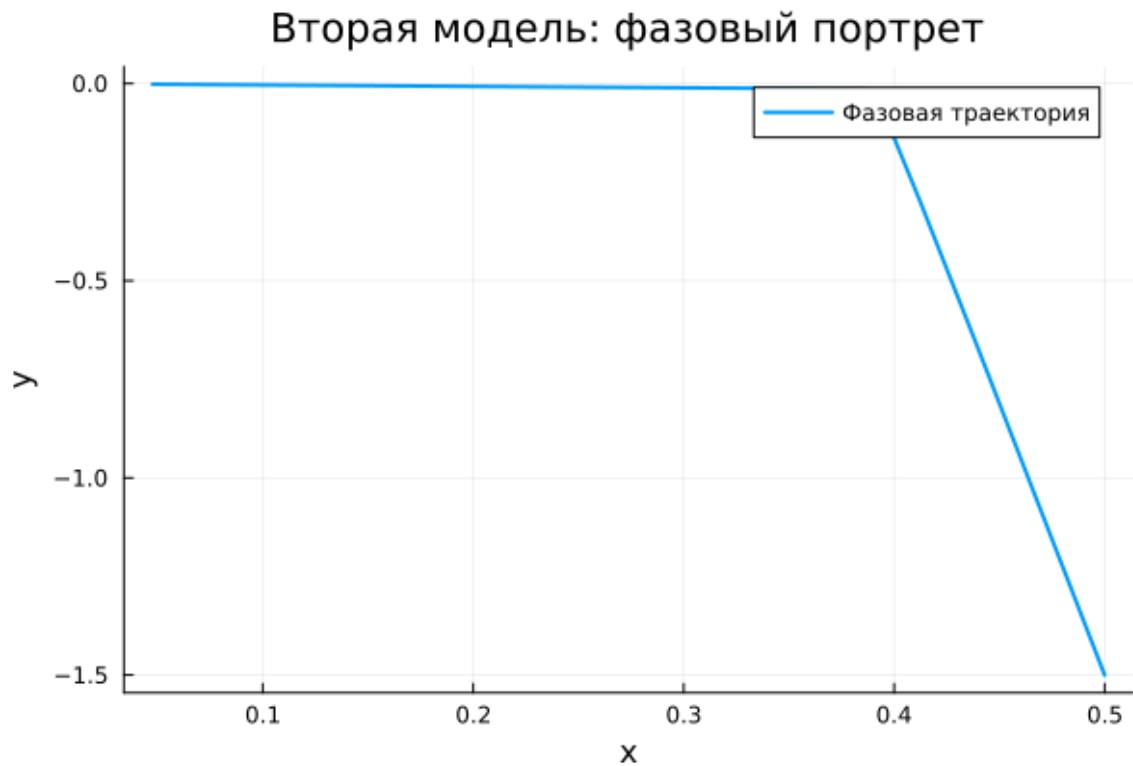


Рисунок 4.4: Вторая модель: фазовый портрет

На графике видно, что колебания быстро исчезают: переменная стремится к нулю уже на начальном участке.

Это обусловлено наличием демпфирования, которое приводит к рассеянию энергии. В результате система переходит в состояние равновесия.

Фазовая траектория стремится к неподвижной точке, что подтверждает затухающий характер процесса.

Таким образом, движение носит апериодический характер и быстро стабилизируется.

4.17.3 Третья модель (`model_type = model3`)

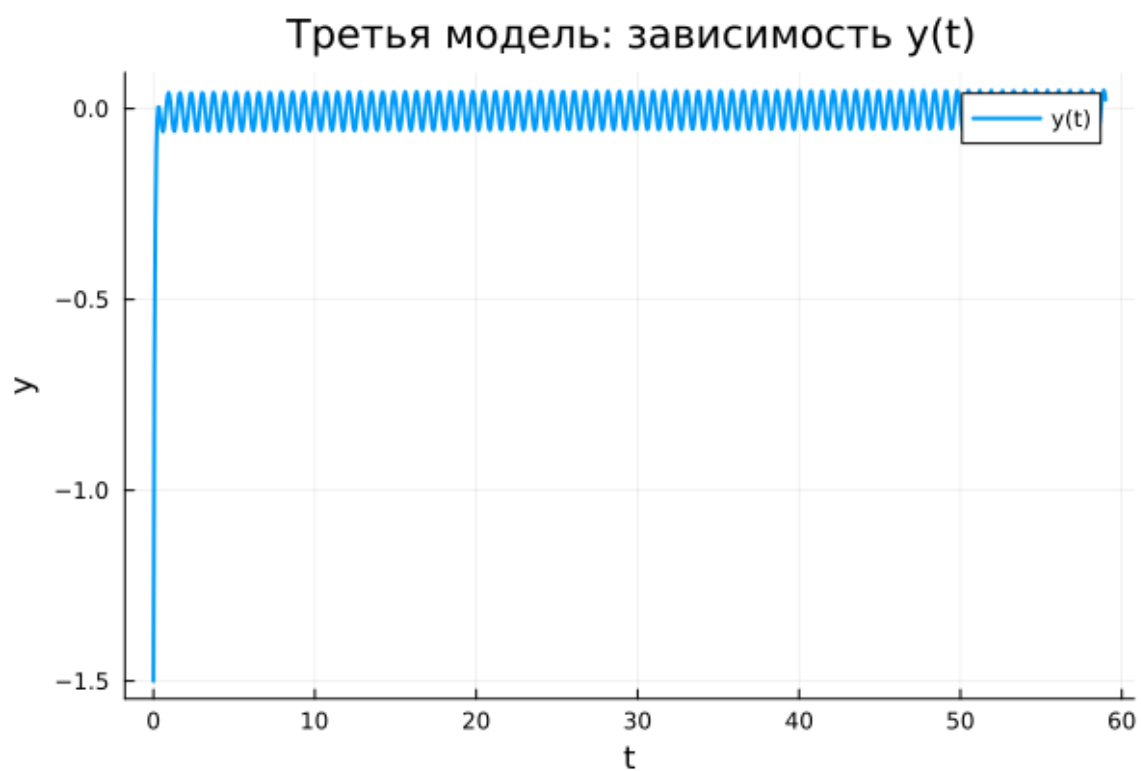


Рисунок 4.5: Третья модель: зависимость $y(t)$

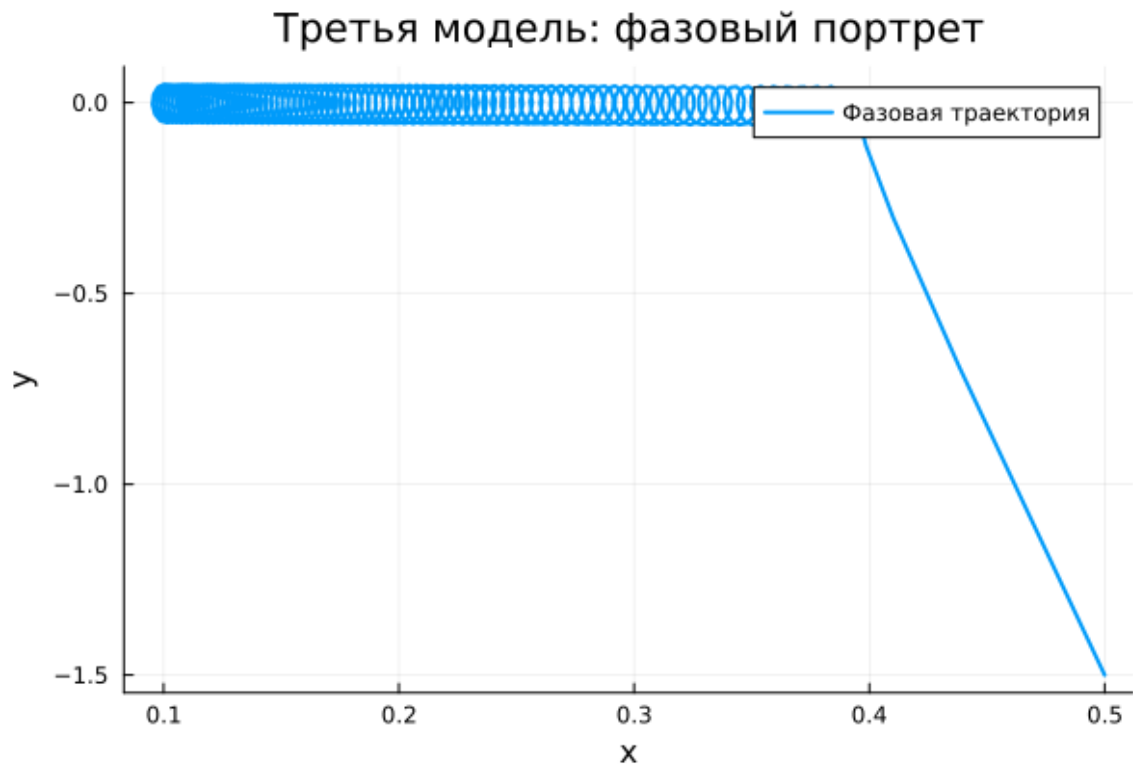


Рисунок 4.6: Третья модель: фазовый портрет

В данном случае после переходного процесса устанавливаются устойчивые колебания небольшой амплитуды.

Несмотря на затухание, внешняя сила поддерживает движение, формируя вынужденные колебания.

Фазовый портрет ограничен небольшой областью около равновесия, что отражает установившийся режим.

Таким образом, система приходит к стационарным вынужденным колебаниям.

4.18 Параметрическое сканирование

4.18.1 Траектории $x(t)$

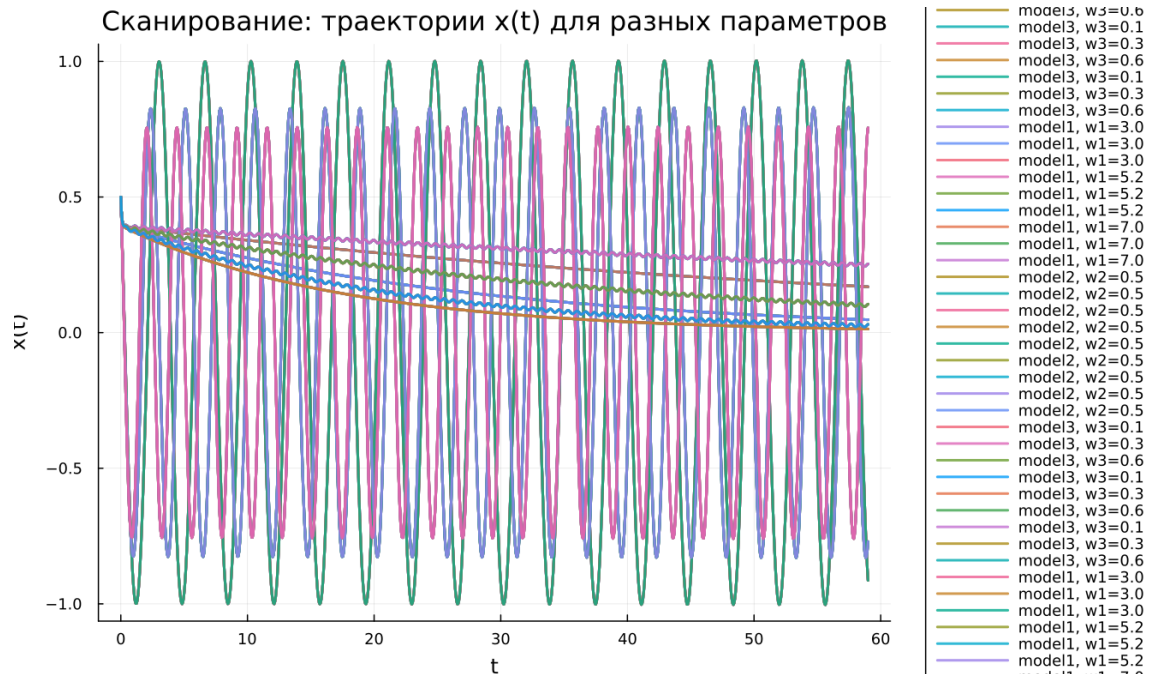


Рисунок 4.7: Сканирование траекторий $x(t)$

Исследование показало:

- в первой модели параметр влияет на частоту колебаний
- во второй — на скорость затухания
- в третьей — на характеристики вынужденного режима

Общие выводы:

- первая модель сохраняет колебания

- третья — формирует устойчивый режим

4.18.2 Траектории $y(t)$

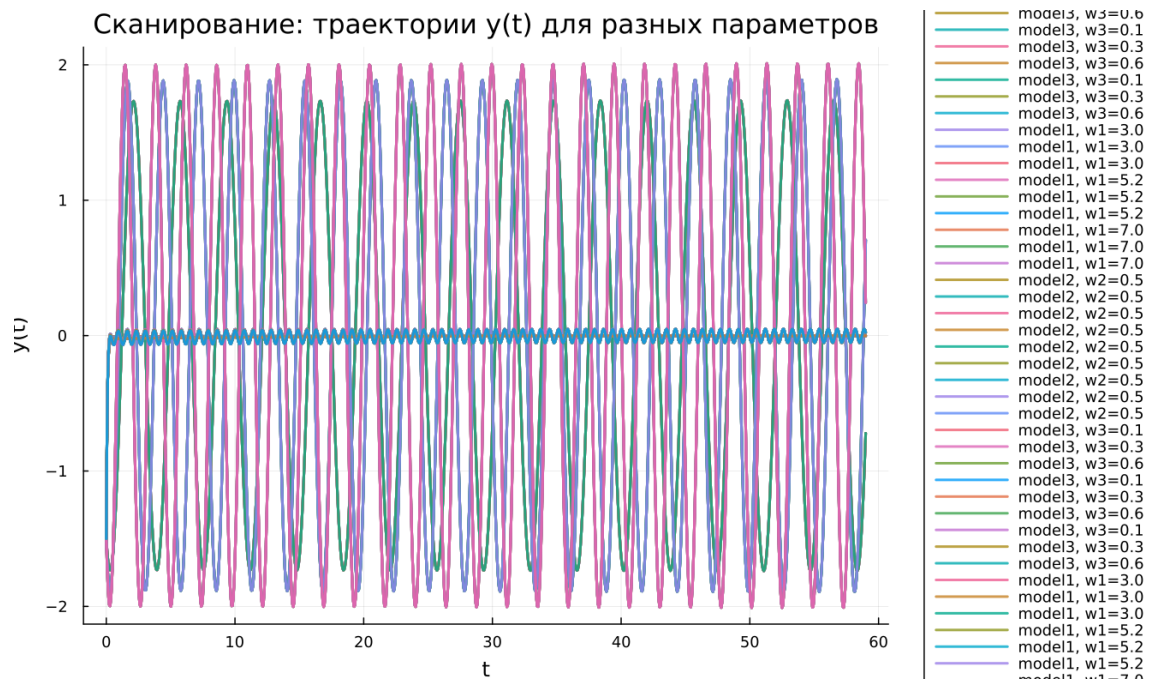


Рисунок 4.8: Сканирование траекторий $y(t)$

Поведение переменной $y(t)$ подтверждает предыдущие выводы:

- устойчивые колебания без затухания
- быстрое затухание
- установившиеся вынужденные колебания

4.19 Время вычислений

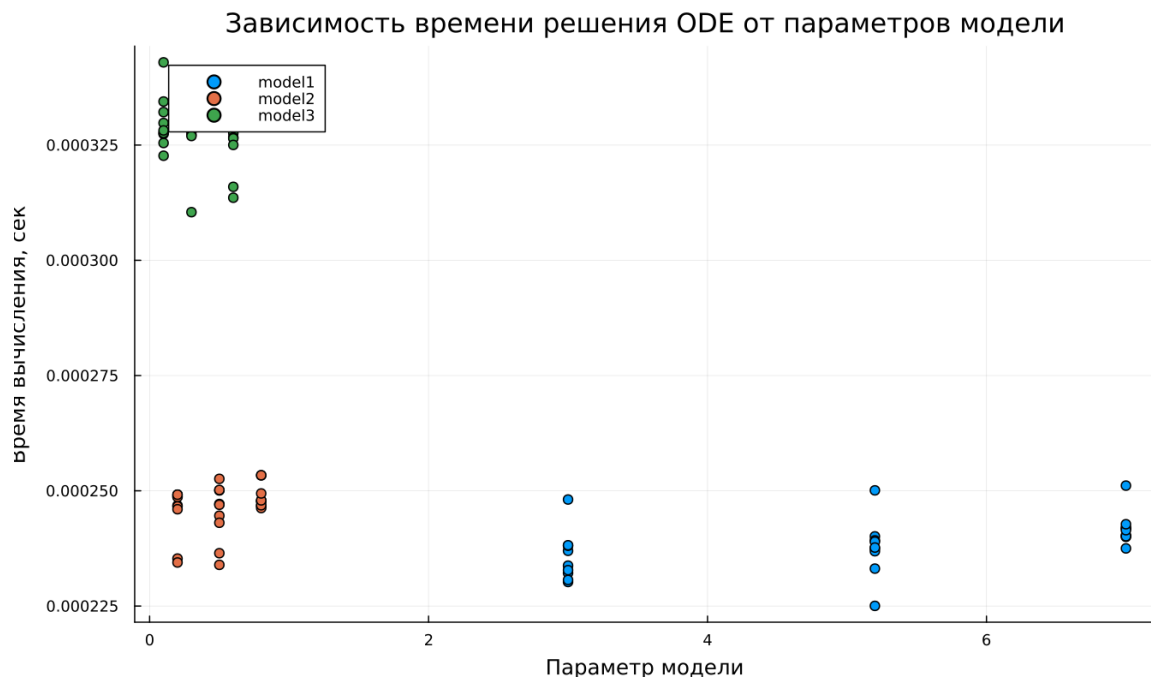


Рисунок 4.9: Зависимость времени вычисления

Анализ показал:

- первая модель вычисляется быстрее всего
- вторая требует немного больше времени
- третья наиболее ресурсоёмкая

Тем не менее, вычисления во всех случаях выполняются быстро.

4.20 Анализ метрики

Рассматривается величина:

$$\text{norm_final} = \sqrt{x(t_{final})^2 + y(t_{final})^2}$$

Результаты:

- первая модель: значение остаётся значительным
- вторая: стремится к нулю
- третья: мало, но не ноль

5 Выводы

1. Незатухающая модель демонстрирует устойчивые гармонические колебания
2. Наличие затухания приводит к переходу системы в состояние покоя
3. Внешняя сила формирует устойчивые вынужденные колебания
4. Параметры существенно влияют на динамику системы
5. Численное моделирование эффективно для всех случаев
6. Метрика `norm_final` отражает различие режимов

Список литературы

1. Гармонический осциллятор
2. Модели колебательных систем